Stock market analysis in SQL

Github Link: https://github.com/KeerthiAnalyst/Stock_Market_project.git

Project Overview

This project is to analyse historical stock market data for six NSE-listed companies using SQL Server. The analysis focuses on stock price trends, moving averages, potential trading signals, daily price movements, and comparative stock performance from January 2015 to July 2018



Objectives

- Analyze historical stock market data for six companies.
- Calculate 20-day and 50-day moving averages.
- Identify potential Buy, Sell, and Hold signals based on moving-average comparisons.
- Combine data from multiple stock tables for comparative analysis.
- Retrieve the latest available stock records using Common Table Expressions (CTEs).

We have six NSE stock datasets covering January 2015 to July 2018, with 889 rows in each table.

Table name	Company
bajaj_auto	Bajaj Auto
eicher_motors	Eicher Motors

hero_motocorp	Hero MotoCorp
infosys	Infosys
tcs	TCS
tvs_motors	TVS Motor

We are analysing stock prices using SQL concepts such as:

- Moving averages — 20-day and 50-day.
- CASE WHEN — identifying Buy, Sell and Hold signals.
- LAG() — comparing current and previous trading days.
- CTE — organising queries into readable steps.
- JOIN — combining stock tables into a master table.
- Latest stock price analysis and insights.

```
-- TASK 1: Trading history in Bajaj Auto

SELECT COUNT(*) AS trading_days, MIN([date]) AS first_day, MAX([date]) AS
last_day
FROM [Bajaj Auto];
OUTPUT:
trading_days first_day    last_day
889          2015-01-01       2018-07-31


-- TASK 2: Eicher Motors' five highest closes

SELECT TOP (5) [date], close_price
FROM dbo.[Eicher Motors]
ORDER BY close_price DESC;
OUTPUT:
date   close_price
2017-09-07   32786.3984375
2017-09-18   32763.849609375
2017-09-08   32628.69921875
2017-09-12   32582.30078125
2017-09-11   32466.75

-- TASK 3: TCS average closing price by year

SELECT YEAR([date]) AS [year],
       ROUND(AVG(CAST(close_price AS DECIMAL(18,4))), 2) AS avg_close
FROM dbo.TCS
GROUP BY YEAR([date])
ORDER BY [year];
OUTPUT:
year   avg_close
2015   2537.390000
2016   2419.000000
2017   2475.360000
2018   2729.120000

-- TASK 4: Find rows with missing deliverable quantity across all six stocks
```

```sql
SELECT 'bajaj_auto' AS stock, [date] FROM dbo.[Bajaj Auto] WHERE
Deliverable_Quantity IS NULL
UNION ALL
SELECT 'eicher_motors', [date] FROM dbo.[Eicher Motors] WHERE
Deliverable_Quantity IS NULL
UNION ALL
SELECT 'hero_motocorp', [date] FROM dbo.Hero_Motocorp WHERE Deliverable_Quantity
IS NULL
UNION ALL
SELECT 'infosys', [date] FROM dbo.Infosys WHERE Deliverable_Quantity IS NULL
UNION ALL
SELECT 'tcs', [date] FROM dbo.tcs WHERE Deliverable_Quantity IS NULL
UNION ALL
SELECT 'tvs_motors', [date] FROM dbo.tvs_motors WHERE Deliverable_Quantity IS
NULL
ORDER BY stock, [date];
OUTPUT:
stock  date
bajaj_auto   2017-08-31
eicher_motors       2015-12-09
hero_motocorp       2015-12-09
infosys      2017-08-31
tcs    2015-12-09
tvs_motors   2015-12-09

-- TASK 5: Bajaj Auto 20-day and 50-day moving averages

DROP TABLE IF EXISTS dbo.bajaj1;
WITH numbered AS (
    SELECT [date], close_price,
           ROW_NUMBER() OVER (ORDER BY [date]) AS rn,
           AVG(CAST(close_price AS DECIMAL(18,4))) OVER (
               ORDER BY [date] ROWS BETWEEN 19 PRECEDING AND CURRENT ROW
           ) AS raw_ma20,
           AVG(CAST(close_price AS DECIMAL(18,4))) OVER (
               ORDER BY [date] ROWS BETWEEN 49 PRECEDING AND CURRENT ROW
           ) AS raw_ma50
    FROM dbo.[Bajaj Auto]
)
SELECT [date], close_price,
       CASE WHEN rn >= 20 THEN ROUND(raw_ma20, 2) END AS ma20,
       CASE WHEN rn >= 50 THEN ROUND(raw_ma50, 2) END AS ma50
INTO dbo.bajaj1
FROM numbered;

SELECT * FROM dbo.bajaj1 ORDER BY [date];
OUTPUT:
date   close_price  ma20   ma50
2015-01-01   2454.10009765625    NULL   NULL
2015-01-02   2453.5 NULL   NULL
2015-01-05   2460.14990234375    NULL   NULL
2015-01-06   2440.35009765625    NULL   NULL
2015-01-07   2447.19995117188    NULL   NULL
2015-01-08   2450.05004882813    NULL   NULL
2015-01-09   2381.5 NULL   NULL
2015-01-12   2333.25      NULL   NULL
2015-01-13   2326.25      NULL   NULL
2015-01-14   2363.64990234375    NULL   NULL

-- TASK 6: Master table with closing prices for all six stocks

DROP TABLE IF EXISTS dbo.master_table;
SELECT b.[date],
```

```
       b.close_price AS bajaj,
       t.close_price AS tcs,
       tv.close_price AS tvs,
       i.close_price AS infosys,
       e.close_price AS eicher,
       h.close_price AS hero
INTO dbo.master_table
FROM dbo.[Bajaj Auto] b
JOIN dbo.tcs t ON t.[date] = b.[date]
JOIN dbo.tvs_motors tv ON tv.[date] = b.[date]
JOIN dbo.infosys i ON i.[date] = b.[date]
JOIN dbo.[Eicher Motors] e ON e.[date] = b.[date]
JOIN dbo.hero_motocorp h ON h.[date] = b.[date];

SELECT * FROM dbo.master_table ORDER BY [date];
OUTPUT:
date   bajaj  tcs    tvs    infosys      eicher hero
2015-01-01   2454.10009765625    2548.19995117188    276.850006103516
       1975.80004882813    15239.150390625     3107.30004882813
2015-01-02   2453.5 2570.85009765625    267.799987792969    2013.44995117188
       15065.5498046875    3097.75
2015-01-05   2460.14990234375    2537.89990234375    271.200012207031
       1996.59997558594    15133.5498046875    3118.64990234375
2015-01-06   2440.35009765625    2446.55004882813    268    1956.84997558594
       14753.099609375     3011.80004882813

-- TASK 7: Golden-cross Buy/Sell/Hold signals for Bajaj Auto

DROP TABLE IF EXISTS dbo.bajaj2;
WITH lagged AS (
    SELECT [date], close_price, ma20, ma50,
           LAG(ma20) OVER (ORDER BY [date]) AS prev_ma20,
           LAG(ma50) OVER (ORDER BY [date]) AS prev_ma50
    FROM dbo.bajaj1
)
SELECT [date], close_price,
       CASE
           WHEN ma20 IS NULL OR ma50 IS NULL
             OR prev_ma20 IS NULL OR prev_ma50 IS NULL THEN 'Hold'
           WHEN ma20 > ma50 AND prev_ma20 <= prev_ma50 THEN 'Buy'
           WHEN ma20 < ma50 AND prev_ma20 >= prev_ma50 THEN 'Sell'
           ELSE 'Hold'
       END AS [signal]
INTO dbo.bajaj2
FROM lagged;

SELECT * FROM dbo.bajaj2 ORDER BY [date];
OUTPUT:
date   close_price  signal
2015-01-01   2454.10009765625    Hold
2015-01-02   2453.5 Hold
2015-01-05   2460.14990234375    Hold
2015-01-06   2440.35009765625    Hold
2015-01-07   2447.19995117188    Hold
2015-01-08   2450.05004882813    Hold
2015-01-09   2381.5 Hold
2015-01-12   2333.25      Hold

-- TASK 8: Count Bajaj Auto signals

SELECT [signal], COUNT(*) AS days
FROM dbo.bajaj2
GROUP BY [signal]
```

```
ORDER BY [signal];
OUTPUT:
signal days
Buy    12
Hold   866
Sell   11

-- TASK 9: Signal on a specific date

SELECT [signal]
FROM dbo.bajaj2
WHERE [date] = '2018-06-21';
OUTPUT:
signal
Buy

-- TASK 10: Signal summary for all six stocks

DROP TABLE IF EXISTS dbo.all_stock_signals;
WITH prices AS (
    SELECT 'Bajaj Auto' AS stock, [date], close_price FROM dbo.[Bajaj Auto]
    UNION ALL SELECT 'Eicher Motors', [date], close_price FROM dbo.[Eicher
Motors]
    UNION ALL SELECT 'Hero Motocorp', [date], close_price FROM dbo.hero_motocorp
    UNION ALL SELECT 'Infosys', [date], close_price FROM dbo.infosys
    UNION ALL SELECT 'TCS', [date], close_price FROM dbo.tcs
    UNION ALL SELECT 'TVS Motors', [date], close_price FROM dbo.tvs_motors
),
ma AS (
    SELECT stock, [date], close_price,
           ROW_NUMBER() OVER (PARTITION BY stock ORDER BY [date]) AS rn,
           AVG(CAST(close_price AS DECIMAL(18,6))) OVER (
               PARTITION BY stock ORDER BY [date] ROWS BETWEEN 19 PRECEDING AND
CURRENT ROW
           ) AS raw_ma20,
           AVG(CAST(close_price AS DECIMAL(18,6))) OVER (
               PARTITION BY stock ORDER BY [date] ROWS BETWEEN 49 PRECEDING AND
CURRENT ROW
           ) AS raw_ma50
    FROM prices
),
ma_ready AS (
    SELECT stock, [date], close_price,
           CASE WHEN rn >= 20 THEN raw_ma20 END AS ma20,
           CASE WHEN rn >= 50 THEN raw_ma50 END AS ma50
    FROM ma
),
lagged AS (
    SELECT stock, [date], close_price, ma20, ma50,
           LAG(ma20) OVER (PARTITION BY stock ORDER BY [date]) AS prev_ma20,
           LAG(ma50) OVER (PARTITION BY stock ORDER BY [date]) AS prev_ma50
    FROM ma_ready
),
sig AS (
    SELECT stock, [date], close_price,
           CASE
               WHEN ma20 IS NULL OR ma50 IS NULL
                 OR prev_ma20 IS NULL OR prev_ma50 IS NULL THEN 'Hold'
               WHEN ma20 > ma50 AND prev_ma20 <= prev_ma50 THEN 'Buy'
               WHEN ma20 < ma50 AND prev_ma20 >= prev_ma50 THEN 'Sell'
               ELSE 'Hold'
           END AS [signal]
    FROM lagged
```

```
),
latest AS (
    SELECT stock, [date] AS last_signal_date, [signal] AS last_signal,
           ROW_NUMBER() OVER (PARTITION BY stock ORDER BY [date] DESC) AS rn
    FROM sig
    WHERE [signal] <> 'Hold'
)
SELECT s.stock,
       SUM(CASE WHEN s.[signal] = 'Buy' THEN 1 ELSE 0 END) AS buys,
       SUM(CASE WHEN s.[signal] = 'Sell' THEN 1 ELSE 0 END) AS sells,
       l.last_signal_date,
       l.last_signal
INTO dbo.all_stock_signals
FROM sig s
LEFT JOIN latest l ON l.stock = s.stock AND l.rn = 1
GROUP BY s.stock, l.last_signal_date, l.last_signal;

SELECT * FROM dbo.all_stock_signals ORDER BY stock;
OUTPUT:
stock  buys   sells  last_signal_date    last_signal
Bajaj Auto   12     11     2018-06-21   Buy
Eicher Motors       6      7      2018-06-06   Sell
Hero Motocorp       9      9      2018-05-22   Sell
Infosys      9      9      2018-05-07   Buy
TCS    12     13     2018-06-05   Sell
TVS Motors   8      8      2018-05-17   Sell

-- TASK 11: First-to-last close percentage change (raw prices; corporate actions
may distort it)

WITH prices AS (
    SELECT 'Bajaj Auto' AS stock, [date], close_price FROM dbo.[Bajaj Auto]
    UNION ALL SELECT 'Eicher Motors', [date], close_price FROM dbo.[Eicher
Motors]
    UNION ALL SELECT 'Hero Motocorp', [date], close_price FROM dbo.hero_motocorp
    UNION ALL SELECT 'Infosys', [date], close_price FROM dbo.infosys
    UNION ALL SELECT 'TCS', [date], close_price FROM dbo.tcs
    UNION ALL SELECT 'TVS Motors', [date], close_price FROM dbo.tvs_motors
),
ends AS (
    SELECT stock, MIN([date]) AS first_date, MAX([date]) AS last_date
    FROM prices GROUP BY stock
)
SELECT p.stock,
       ROUND(CAST(firstp.close_price AS DECIMAL(18,4)), 2) AS first_close,
       ROUND(CAST(lastp.close_price AS DECIMAL(18,4)), 2) AS last_close,
       ROUND(100.0 * (CAST(lastp.close_price AS DECIMAL(18,6)) /
                      NULLIF(CAST(firstp.close_price AS DECIMAL(18,6)), 0) - 1),
1) AS pct_change
FROM ends e
JOIN prices firstp ON firstp.stock = e.stock AND firstp.[date] = e.first_date
JOIN prices lastp ON lastp.stock = e.stock AND lastp.[date] = e.last_date
JOIN (SELECT DISTINCT stock FROM prices) p ON p.stock = e.stock
ORDER BY pct_change DESC;
OUTPUT:
stock  first_close  last_close   pct_change
TVS Motors   276.8500     517.4500     86.9000000000000000
Eicher Motors       15239.1500   27820.9500   82.6000000000000000
Bajaj Auto   2454.1000    2700.7000    10.0000000000000000
Hero Motocorp       3107.3000    3293.8000    6.0000000000000000
TCS    2548.2000    1941.2500    -23.8000000000000000
Infosys      1975.8000    1365.0000    -30.9000000000000000
```

```sql
-- TASK 12: Worst day-over-day percentage move for each stock

WITH prices AS (
    SELECT 'Bajaj Auto' AS stock, [date], close_price FROM dbo.[Bajaj Auto]
    UNION ALL SELECT 'Eicher Motors', [date], close_price FROM dbo.[Eicher
Motors]
    UNION ALL SELECT 'Hero Motocorp', [date], close_price FROM dbo.Hero_Motocorp
    UNION ALL SELECT 'Infosys', [date], close_price FROM dbo.infosys
    UNION ALL SELECT 'TCS', [date], close_price FROM dbo.tcs
    UNION ALL SELECT 'TVS Motors', [date], close_price FROM dbo.tvs_motors
),
moves AS (
    SELECT stock, [date], close_price,
           LAG(close_price) OVER (PARTITION BY stock ORDER BY [date]) AS
prev_close
    FROM prices
),
ranked AS (
    SELECT stock, [date], close_price,
           ROUND(100.0 * (CAST(close_price AS DECIMAL(18,6)) /
                          NULLIF(CAST(prev_close AS DECIMAL(18,6)), 0) - 1), 1)
AS pct_move,
           ROW_NUMBER() OVER (
               PARTITION BY stock
               ORDER BY (CAST(close_price AS DECIMAL(18,6)) /
                         NULLIF(CAST(prev_close AS DECIMAL(18,6)), 0) - 1) ASC
           ) AS rn
    FROM moves
    WHERE prev_close IS NOT NULL
)
SELECT stock, [date], close_price, pct_move
FROM ranked
WHERE rn = 1
ORDER BY pct_move ASC;
OUTPUT:
stock  date   close_price  pct_move
TCS    2018-05-31   1744.80004882813    -50.4000000000000000
Infosys      2015-06-15   991.099975585938    -49.9000000000000000
TVS Motors   2016-05-03   287.899993896484    -9.7000000000000000
Eicher Motors       2015-08-24   18571.75     -9.3000000000000000
Bajaj Auto   2015-08-24   2188.44995117188    -9.1000000000000000
Hero Motocorp       2018-07-23   3163.89990234375    -6.2000000000000000

-- TASK 13: Adjust TCS and Infosys for 1:1 bonus issues, then compare first and
last closes.
-- VERIFY these event dates against your Task 12 findings and source data before
using results.

WITH adjusted AS (
    SELECT 'TCS' AS stock, [date],
           CASE WHEN [date] < '2018-06-28'
                THEN CAST(close_price AS DECIMAL(18,6)) / 2.0
                ELSE CAST(close_price AS DECIMAL(18,6)) END AS adj_close
    FROM dbo.tcs
    UNION ALL
    SELECT 'Infosys', [date],
           CASE WHEN [date] < '2015-06-15'
                THEN CAST(close_price AS DECIMAL(18,6)) / 2.0
                ELSE CAST(close_price AS DECIMAL(18,6)) END AS adj_close
    FROM dbo.infosys
),
ends AS (
    SELECT stock, MIN([date]) AS first_date, MAX([date]) AS last_date
```

```
    FROM adjusted GROUP BY stock
)
SELECT a.stock,
       ROUND(100.0 * (
           MAX(CASE WHEN a.[date] = e.last_date THEN a.adj_close END) /
           NULLIF(MAX(CASE WHEN a.[date] = e.first_date THEN a.adj_close END), 0)
- 1
       ), 1) AS adjusted_pct_change
FROM adjusted a
JOIN ends e ON e.stock = a.stock
GROUP BY a.stock
ORDER BY a.stock;
OUTPUT:
stock  adjusted_pct_change
Infosys      38.200000000000
TCS    52.400000000000

-- FINAL VALIDATION: row counts for the six source tables

SELECT 'bajaj_auto' AS stock, COUNT(*) AS rows_count FROM dbo.[Bajaj Auto]
UNION ALL SELECT 'eicher_motors', COUNT(*) FROM dbo.[Eicher Motors]
UNION ALL SELECT 'hero_motocorp', COUNT(*) FROM dbo.hero_motocorp
UNION ALL SELECT 'infosys', COUNT(*) FROM dbo.infosys
UNION ALL SELECT 'tcs', COUNT(*) FROM dbo.tcs
UNION ALL SELECT 'tvs_motors', COUNT(*) FROM dbo.tvs_motors;
OUTPUT:
stock  rows_count
bajaj_auto   889
eicher_motors       889
hero_motocorp       889
infosys      889
tcs    889
tvs_motors   889
```

Key Insights

1. **Stock Price Trends:** The analysis of six NSE-listed companies — Bajaj Auto, Eicher Motors, Hero MotoCorp, Infosys, TCS, and TVS Motor — helps identify changes in stock prices over the period from January 2015 to July 2018.
2. **Moving Average Analysis:** The 20-day and 50-day moving averages help smooth daily price fluctuations and identify short-term and longer-term price trends.
3. **Buy and Sell Signals:** By comparing moving averages and using SQL window functions such as LAG(), the analysis identifies potential bullish and bearish signals. These signals help understand changes in market momentum but do not guarantee future price movements.
4. **Daily Price Comparison:** Comparing current and previous trading-day prices helps identify increases, decreases, and changes in stock performance.
5. **Comparative Stock Analysis:** Combining the six stock tables into a master table makes it easier to compare companies and analyse their price movements over the same period.
6. **SQL-Based Analysis:** SQL functions, including aggregate functions, Common Table Expressions (CTEs), joins, and window functions, enable efficient data processing and structured analysis of historical stock market data.

Conclusion

The Stock Market Analysis project demonstrates how SQL can be used to analyse historical stock market data and identify meaningful patterns in stock price movements. The project covers six NSE-listed companies over the period from January 2015 to July 2018.

Using moving averages, daily price comparisons, window functions, and combined stock tables, the analysis provides insights into price trends, market momentum, and differences in stock performance.

This project strengthened my practical understanding of SQL concepts, including CTEs, joins, aggregate functions, and window functions, while demonstrating their application to financial data analysis.

Overall, the project shows how structured analysis of historical stock data can support better understanding of market trends and informed financial research. However, historical patterns and technical signals do not guarantee future stock performance.